



Desarrollo de Aplicaciones con Web Frameworks DWF404 G01T

Proyecto de Cátedra – Primera Etapa

Sistema Digital de Gestión de Inventario y Ventas

Alumnos

Carnét

Odaly Rachel Cruz Franco

CF252175

Christopher Steven Jovel Beltrán

JB251834

Giovanni Alberto Ruano Martínez

RM250065

Elisabet Beatriz Marroquín González

MG251360

Docente: Ingeniero Bladimir Díaz Campos

Fecha de entrega: 02/03/2026

Introducción.....	3
Justificación.....	4
Alcance de la primera etapa.....	4
3. Planteamiento del Problema.....	5
3.1 Descripción del Problema.....	5
3.2 Contexto Organizacional o Social.....	5
3.3 Necesidad Identificada.....	6
4. Objetivos del Proyecto.....	7
4.1 Objetivo General.....	7
4.2 Objetivos Específicos.....	7
5. Alcances y Limitaciones.....	7
5.1 Alcances Técnicos.....	7
5.2 Limitaciones del Proyecto.....	8
6. Marco Teórico.....	9
6.1 Arquitectura Cliente–Servidor.....	9
6.2 Arquitectura en Capas.....	10
1. Capa de Presentación (Controller).....	10
2. Capa de Negocio (Service).....	10
3. Capa de Persistencia (Repository).....	10
4. Capa de Datos.....	10
6.3 Patrón MVC.....	11
Modelo (Model).....	11
Vista (View).....	11
Controlador (Controller).....	11
6.4 Introducción a Spring Boot.....	12
6.5 RESTful APIs.....	13
6.6 JSON y Serialización.....	14
7. Metodología de Desarrollo.....	15
7.1 Modelo de Desarrollo Seleccionado (Ágil / Scrum / Incremental).....	15
7.2 Roles del Equipo.....	16
8. Análisis de Requisitos.....	17
8.1 Requisitos Funcionales.....	17
8.2 Requisitos No Funcionales.....	17
8.3 Historias de Usuario.....	18
8.4 Casos de Uso.....	19
Diagrama de Casos de Uso.....	20

Introducción

El presente proyecto consiste en el desarrollo de una Sistema de ventas y administración de inventario siendo diseñada para agilizar la experiencia de compras en tiendas físicas mediante el modelo de autoservicio asistido. El sistema estará dividido en dos interfaces : una interfaz dirigida al cliente que funcionará como un catálogo interactivo y un panel administrativo destinado al control operativo destinado al personal de la tienda. El objetivo central es que el sistema permita que el usuario final gestione su propio pedido sin depender de la asistencia inmediata de un vendedor

Desde la perspectiva del cliente el sistema permite explorar categorías de productos, verifica la disponibilidad real de tallas, colores y estructura de orden autónoma de un pedido. Al finalizar la selección, el sistema genera un identificador de orden único (un ticket) que vincula la intención de compra digital con la transacción física.

En el ámbito administrativo, la plataforma centraliza la gestión de ventas y la actualización de existencias en una sola base de datos. Los empleados disponen de una interfaz dedicada para procesar las órdenes entrantes, confirmar pagos y realizar el despacho de mercancía, lo que garantiza que el inventario se descuente de forma automática y precisa.

Justificación

La implementación de este sistema se realizara por que hoy en dia hay muchas personas que al momento de realizar una compra en una tienda física suelen frustrarse o disgustarse e irse sin hacer su compra por estar en la espera a que un vendedor los atienda o les confirme si hay stock de un producto. Queremos eliminar esa frustración dándole al cliente el control total de su búsqueda. Si el cliente puede ver por sí mismo qué hay disponible y armar su carrito en una pantalla, el proceso de venta se vuelve mucho más ágil y el personal de la tienda puede enfocarse en cobrar y entregar los productos sin errores.

Este sistema ayuda a que el negocio esté mucho mejor organizado. Al conectar la pantalla del cliente con el inventario del administrador, se evitan confusiones como vender algo que ya no queda en bodega o tener precios desactualizados. Es una solución práctica porque no requiere instalar complejos sistemas de pago con tarjeta en las pantallas de consulta; simplemente digitaliza el proceso de selección y deja el pago en la caja, lo que hace que la implementación sea más sencilla, económica y eficiente para cualquier tienda.

Alcance de la primera etapa

El alcance de esta primera etapa se limita al **diseño integral y la definición técnica** del sistema. El objetivo es dar una propuesta sólida que resuelva el flujo de "compra asistida" desde que el cliente toca la pantalla hasta que el cajero confirma la entrega. El proyecto alcanzará la definición de la arquitectura del sistema, y la estructuración de la base de datos que permitirá que el inventario se actualice en tiempo real.

3. Planteamiento del Problema

3.1 Descripción del Problema

En las tiendas de ropa y zapatos tradicionales, comprar algo depende casi por completo de que un vendedor esté libre para atendernos. Cuando un cliente entra, solo puede elegir entre los productos que están a la vista en los estantes. En los días en que la tienda está muy llena, como los fines de semana o días de pago, esto se convierte en un gran problema. El cliente tiene que buscar a un empleado, hacer fila o esperar su turno para ser atendido, y luego preguntarle si hay un modelo específico en su talla o en el color que busca. Después de eso, le toca esperar a que el empleado vaya hasta la bodega a revisar si el producto realmente está ahí.

Esta rutina de ir y venir de la bodega quita mucho tiempo. En lugar de estar cobrando o ayudando a otros clientes de forma rápida, el personal pasa gran parte de su día buscando tallas. Para el cliente, esta espera es cansada y corta la emoción de comprar. Muchas veces, después de esperar varios minutos, el empleado regresa para decirle que "no hay en su talla". Si el cliente quiere ver otro modelo, tiene que empezar todo el proceso desde cero, lo que causa tanta frustración que muchas personas prefieren irse de la tienda sin comprar nada.

Además, el espacio de cualquier tienda física es pequeño, por lo que es imposible tener todos los zapatos o prendas a la vista. Esto significa que muchos productos o promociones se quedan guardados en la bodega sin que el cliente se entere de que existen. Hoy en día, las personas están acostumbradas a comprar por internet o usar aplicaciones donde pueden ver todo lo que hay disponible al instante. Al obligar al cliente a comprar de esta forma lenta y dependiendo de un vendedor, las tiendas físicas dan un servicio anticuado y terminan perdiendo ventas.

3.2 Contexto Organizacional o Social

Actualmente las personas valoran mucho hacer las cosas rápido y por su cuenta. En lugares como los restaurantes de comida rápida o los cines, ya es muy normal ver pantallas táctiles donde uno mismo hace su pedido sin tener que hablar con un cajero. Sin embargo, en las tiendas de ropa y zapatos, usar este tipo de tecnología todavía es muy raro.

Desde el punto de vista social, el cliente actual quiere lo mejor de los dos mundos: la facilidad de comprar en línea (donde puede ver fotos de los zapatos, filtrar por su talla y saber si hay stock al instante) y la ventaja de la tienda física donde se lleva el producto a casa ese mismo día..

Por el lado de las tiendas tienen el gran reto de aprovechar mejor el tiempo de sus empleados. Actualmente, los trabajadores pasan demasiadas horas haciendo tareas repetitivas, como ir a la bodega a revisar si hay una talla de zapato, en lugar de estar cobrando rápido en la caja, ordenando la tienda o dando una atención al cliente de mayor calidad. Las tiendas necesitan modernizarse para no quedarse atrás frente a las compras por internet y para hacer que la visita de sus clientes sea mucho más ágil y agradable.

3.3 Necesidad Identificada

Al ver la frustración de los clientes cuando les toca esperar y notar que los empleados no dan abasto en los días de tienda llena, la solución se vuelve muy evidente: la tienda necesita darle el control de la compra al cliente a través de un Sistema Digital de Gestión de Inventario y Ventas.

La necesidad principal es instalar una pantalla táctil amigable e intuitiva con una interfaz cómoda, donde los contrastes y el color de fondo inviten a usarla sin complicaciones para que cualquier persona pueda acercarse, buscar los zapatos o la ropa que le gustan, y revisar por su cuenta si hay en su talla. La idea es que el cliente arme su propio carrito a su ritmo, sin presiones, y que la pantalla simplemente le imprima un ticket para ir directo a la caja a pagar y recoger sus cosas.

Por otro lado, existe una necesidad igual de importante para el equipo de empleados. Necesitamos construirles un panel de control en la computadora donde ya no tengan que adivinar qué hay en la bodega. Un sistema donde puedan actualizar precios con un clic, subir fotos y, sobre todo, ver en tiempo real los tickets que los clientes están generando en la pantalla para ir preparando los zapatos antes de que la persona llegue a la caja.

4. Objetivos del Proyecto

4.1 Objetivo General

Desarrollar un sistema integral de autoconsulta y administración para una tienda de ropa y zapatos, con el propósito de devolverle al cliente el control de su experiencia de compra mediante un pantalla interactiva, y al mismo tiempo, facilitar el trabajo diario de los empleados al automatizar el control del inventario y la gestión de ventas.

4.2 Objetivos Específicos

Eliminar las filas frustrantes: Agilizar el proceso final de compra al permitir que el cliente arme su propio pedido en la pantalla y reciba un ticket de confirmación, de forma que solo pase a la caja directamente a pagar y recoger sus compras, ahorrando tiempo

Darle el control y mayor comodidad al cliente: Crear una interfaz visualmente agradable y muy fácil de usar permitiendo personalizar detalles como las tallas y colores disponibles sin sentirse apurado ni presionado por un vendedor.

Disminuir el trabajo diario del personal: Construir un panel de control interno que sea una herramienta de apoyo para los empleados, donde puedan actualizar precios, agregar nuevos zapatos y revisar lo que hay en bodega con unos pocos clics, sin tener que abandonar la caja ni hacer viajes cansados a la bodega.

Evitar decepciones y asegurar el inventario: Conectar con la información que el cliente ve en la pantalla interactiva con las cajas de zapatos reales que hay en la tienda. Esto se actualizará al instante para evitar que una persona se ilusione con una prenda o zapato que ya se agotó.

5. Alcances y Limitaciones

5.1 Alcances Técnicos

Para el cliente (Pantalla Interactiva): Entregaremos una interfaz táctil donde las personas podrán navegar por todo el catálogo de ropa y zapatos. Podrán buscar por categorías, ver fotografías, confirmar si su talla y color favorito están disponibles, y armar su pedido a su ritmo. Al terminar, la pantalla interactiva generará un "Ticket de Pre-compra" con un número de orden.

Para el empleado (Panel Administrativo): Desarrollaremos un portal web seguro donde el personal, usando su usuario y contraseña, podrá agregar nuevos zapatos, actualizar precios, registrar cuántas tallas hay físicamente en la bodega y ver en tiempo real los tickets que los clientes están armando en la tienda.

Sincronización al instante: El sistema conectará el inventario con la pantalla interactiva de forma automática. Si en la caja física se vende el último par de zapatos talla 40, esa talla desaparecerá de la pantalla inmediatamente para evitar que alguien más la pida por error y se lleve una decepción.

Reinicio automático por inactividad: Para proteger la privacidad del cliente y mantener el flujo en la tienda, si una persona abandona la pantalla interactiva a mitad de su compra, el sistema borrará automáticamente el carrito y restablecerá las configuraciones visuales después de un par de minutos sin tocar la pantalla, dejándola limpia y lista para el siguiente usuario.

5.2 Limitaciones del Proyecto

Complejidad en la Arquitectura de Seguridad: Debido a que nuestra experiencia con **Spring Security** es inicial, el sistema se limita a una autenticación de roles básica (Administrador/Cajero). No contamos con el conocimiento avanzado para implementar medidas de seguridad de grado bancario, cifrado de extremo a extremo complejo o protocolos de recuperación de cuentas mediante servicios externos, lo que restringe el despliegue a un entorno controlado y privado.

Optimización y Refactorización del Código: Al ser estudiantes en formación, la lógica del backend en **Java/Spring Boot** puede no estar optimizada para manejar miles de peticiones simultáneas. Carecemos del conocimiento en patrones de diseño avanzados o "tunning" de la Máquina Virtual de Java (JVM), lo que limita el rendimiento del sistema si se escala fuera de un entorno de prueba local

6. Marco Teórico

6.1 Arquitectura Cliente–Servidor

La arquitectura Cliente–Servidor es un modelo de diseño de software distribuido en el cual las funciones del sistema se dividen entre dos componentes principales: el cliente, que solicita servicios, y el servidor, que los proporciona. Este modelo permite centralizar el procesamiento de datos y la lógica de negocio en un servidor, mientras que el cliente se enfoca en la interacción con el usuario.

En este modelo:

- El **cliente** envía solicitudes mediante protocolos de comunicación como HTTP.
- El **servidor** recibe la solicitud, la procesa, accede a la base de datos si es necesario y devuelve una respuesta estructurada.

Una característica fundamental de este modelo es la separación de responsabilidades, lo que facilita:

- Escalabilidad horizontal (añadiendo más servidores).
- Seguridad (control centralizado de autenticación y autorización).
- Mantenimiento independiente del frontend y backend.
- Reutilización de servicios mediante APIs.

En el presente proyecto, el sistema funciona bajo esta arquitectura:

- La interfaz y el panel administrativo actúan como clientes.
- El servidor backend desarrollado con Spring Boot procesa las solicitudes.
- La base de datos almacena la información persistente relacionada con productos, inventario y órdenes.

6.2 Arquitectura en Capas

La arquitectura en capas es un patrón estructural que organiza el sistema en niveles jerárquicos con responsabilidades claramente definidas. Cada capa cumple una función específica y se comunica únicamente con la capa inmediatamente inferior o superior. En aplicaciones empresariales desarrolladas con Spring Boot, se implementa comúnmente una arquitectura de tres o cuatro capas:

1. Capa de Presentación (Controller)

Se encarga de recibir las solicitudes HTTP del cliente, validarlas y delegar el procesamiento a la capa de negocio. No debe contener lógica compleja, sino actuar como intermediario.

2. Capa de Negocio (Service)

Contiene las reglas del negocio, validaciones, cálculos y procesos transaccionales. Por ejemplo:

- Validar disponibilidad de stock antes de confirmar una orden.
- Calcular el total de una compra.
- Aplicar descuentos o promociones.

3. Capa de Persistencia (Repository)

Se encarga del acceso a la base de datos mediante herramientas como Spring Data JPA, que permite trabajar con entidades Java sin necesidad de escribir consultas SQL complejas manualmente.

4. Capa de Datos

Corresponde al sistema gestor de base de datos (por ejemplo, PostgreSQL o MySQL), donde se almacena la información estructurada.

Las ventajas de este enfoque incluyen:

- Mayor organización del código.
- Reducción del acoplamiento.

- Facilidad para realizar pruebas unitarias.
- Escalabilidad estructural.

6.3 Patrón MVC

El patrón Modelo–Vista–Controlador (MVC) es un patrón de diseño que divide la aplicación en tres componentes principales con el objetivo de separar la lógica de negocio de la interfaz de usuario.

Modelo (Model)

Representa los datos del sistema y su lógica interna. En este proyecto, incluye entidades como:

- Producto
- Categoría
- Inventario
- Orden
- Detalle de Orden

Estas entidades están mapeadas mediante JPA y gestionadas por el backend.

Vista (View)

Es la interfaz gráfica que interactúa con el usuario. Puede estar desarrollada en tecnologías frontend modernas como React o mediante motores de plantillas como Thymeleaf.

Controlador (Controller)

Recibe las solicitudes del usuario, interactúa con el modelo a través de la capa de servicio y devuelve una respuesta. En Spring Boot, los controladores se implementan mediante anotaciones como `@RestController`.

El uso de MVC permite:

- Separar claramente responsabilidades.
- Facilitar la escalabilidad del sistema.
- Mejorar la mantenibilidad.
- Permitir que el frontend evolucione sin afectar la lógica de negocio.

6.4 Introducción a Spring Boot

Spring Boot es un framework de desarrollo basado en el ecosistema Spring que simplifica la creación de aplicaciones Java empresariales mediante configuración automática y convenciones predefinidas.

Spring Boot surge como una solución para reducir la complejidad del framework Spring tradicional, ofreciendo:

- Configuración automática basada en dependencias.
- Servidor embebido (Tomcat).
- Integración con bases de datos.
- Manejo de seguridad.
- Soporte para microservicios.
- Desarrollo rápido de APIs REST.

Entre sus principales características se encuentran:

- Uso de anotaciones para configuración.
- Gestión automática de dependencias mediante Maven o Gradle.
- Soporte para pruebas unitarias.
- Manejo de transacciones.

En este proyecto, Spring Boot permite:

- Exponer servicios REST para consulta y creación de órdenes.
- Gestionar inventario en tiempo real.
- Implementar autenticación y control de roles.
- Integrar fácilmente con bases de datos relacionales.

6.5 RESTful APIs

REST (Representational State Transfer) es un estilo arquitectónico para el diseño de servicios web que utilizan el protocolo HTTP como medio de comunicación.

Una API RESTful se caracteriza por:

- Comunicación sin estado (stateless).
- Uso de métodos HTTP estándar (GET, POST, PUT, DELETE).
- Identificación de recursos mediante URLs.
- Representación de datos en formatos ligeros como JSON.

En aplicaciones desarrolladas con Spring Boot, las APIs REST se implementan mediante controladores anotados con `@RestController`, permitiendo exponer endpoints como:

- `/productos`
- `/ordenes`
- `/inventario`
- `/usuarios`

Las APIs REST son fundamentales en este proyecto porque:

- Permiten la comunicación entre el frontend (Tienda) y el backend.
- Facilitan la integración futura con aplicaciones móviles.
- Permiten escalar el sistema hacia una arquitectura de microservicios.
- Garantizan interoperabilidad entre diferentes plataformas.

Además, al ser stateless, cada petición contiene toda la información necesaria para ser procesada, lo que mejora la escalabilidad y confiabilidad del sistema.

6.6 JSON y Serialización

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos ampliamente utilizado en aplicaciones web y APIs REST.

Su estructura se basa en pares clave–valor y es fácil de leer tanto para humanos como para máquinas.

Ejemplo de estructura JSON:

```
4  {  
5    "nombre": "Tenis Nike Air",  
6    "precio": 89.99,  
7    "talla": "42",  
8    "color": "Blanco"  
9  }
```

En aplicaciones desarrolladas con Spring Boot, la conversión entre objetos Java y JSON se realiza mediante un proceso conocido como serialización y deserialización, generalmente utilizando la biblioteca Jackson integrada automáticamente.

- **Serialización:** Conversión de un objeto Java a formato JSON para enviarlo al cliente.
- **Deserialización:** Conversión de JSON recibido en una solicitud HTTP a un objeto Java.

Este mecanismo permite:

- Intercambio eficiente de datos.
- Independencia tecnológica entre frontend y backend.
- Integración con múltiples plataformas.

En el sistema propuesto, JSON es el formato principal para:

- Enviar catálogos de productos.
- Crear órdenes.
- Actualizar inventario.
- Gestionar usuarios.

7. Metodología de Desarrollo

7.1 Modelo de Desarrollo Seleccionado (Ágil / Scrum / Incremental)

El modelo de desarrollo que utilizaremos es el **Incremental** ya que se apoya en una metodología **Ágil**. En lugar de intentar construir todo el sistema y esperar hasta el final para ver si funciona, lo vamos a ir armando por módulos funcionales.

El desarrollo se dividirá en etapas cortas:

- **Primer Paso (La Base):** Nos enfocaremos en que el administrador pueda subir productos, tallas y colores a una base de datos central.
- **Segundo Paso (La Experiencia del Cliente):** Crearemos la interfaz de la pantalla táctil para que el cliente pueda navegar por las categorías y armar su carrito de forma autónoma.
- **Tercer Paso (El Cierre del Círculo):** Desarrollaremos la conexión entre el ticket que genera el cliente y la pantalla de "Órdenes Activas" que ve el cajero para procesar el pago.
- **Cuarto Paso (Pulido y Extras):** Una vez que el flujo principal funcione, añadiremos las funciones de promociones, descuentos y reportes de ventas para el administrador.

7.2 Roles del Equipo

Rol	Responsabilidades Principales	Enfoque en el Proyecto
Desarrollador Backend (Christopher Steven Jovel Beltran)	Construir la lógica del sistema con Spring Boot, diseñar la base de datos y crear las APIs que conectan todo el sistema.	Asegurar que el inventario se actualice correctamente y que las órdenes se guarden de forma segura.
Desarrollador Frontend (Odaly Rachel Cruz)	Crear las interfaces visuales donde interactúa el usuario, asegurando que los datos del backend se vean en tiempo real.	Desarrollar la pantalla táctil para el cliente y el panel web privado para el empleado.
Diseñador (Elisabet Beatriz Marroquín González)	Diseñar el aspecto visual de la plataforma, los botones, colores y el flujo de navegación para que sea intuitivo.	Lograr que un cliente pueda armar su pedido en pocos pasos y sin necesidad de ayuda.
Analista de Sistemas (Giovanni Alberto Ruano Martínez)	Validar que todos los procesos diseñados funcionen sin errores y que la información sea coherente.	Verificar que no haya fallos lógicos entre lo que el cliente pide y lo que el administrador recibe.

8. Análisis de Requisitos

8.1 Requisitos Funcionales

El Cliente: Al llegar a la tienda, el cliente se encontrará con nuestro sistema a través de una pantalla interactiva muy cómoda aquí podrá explorar categorías como zapatos, camisas o pantalones. Cuando encuentre algo que le guste, por ejemplo unos tenis Nike, el sistema consultará la bodega y le mostrará *solo* las tallas que realmente están disponibles. El cliente irá guardando sus artículos en un carrito virtual y, al terminar, presionará "Finalizar Pedido". El sistema generará un número de orden único (por ejemplo, el #105) y lo mostrará en grande en la pantalla para que el cliente pase a la caja a pagar, sin necesidad de imprimir papel.

El Cajero Panel de Órdenes Activas: Para el empleado en caja, el sistema funcionará como una bandeja de entrada. Tendrá una pantalla llamada "Órdenes Activas" donde se mostrarán todos los pedidos que los clientes han generado pero que todavía no pagan. Cuando el cliente dicte su número de orden, el cajero lo buscará y marcará que ya le pagaron. En ese instante, el sistema hará algo muy importante: descontará automáticamente esos productos del inventario general para que al final del día no haya problemas ni ventas fantasma.

El Administrador Gestión de Tienda: Esta persona, que generalmente es el dueño o encargado, tendrá funciones más de fondo. Desde su sistema podrá agregar productos nuevos con su foto, descripción, precio y stock inicial. También podrá cambiar precios rápidamente cuando haya ofertas, creando promociones como "20% de descuento este fin de semana". Aunque estas funciones no se usen todos los días, están diseñadas para estar siempre disponibles y ser muy fáciles de usar.

8.2 Requisitos No Funcionales

Usabilidad y Pruebas Reales: La pantalla interactiva del cliente tiene que ser sumamente fácil de usar. Aunque suene obvio, es clave especificarlo porque no queremos que el cliente necesite un manual para entenderla. Los botones serán grandes, las imágenes claras, Todo debe ser tan intuitivo que cualquier persona, sin importar su edad o experiencia tecnológica, pueda usarlo sin ayuda. Para garantizar esto, realizaremos pruebas reales invitando a personas de distintas edades a usar la pantalla para ver si logran armar un pedido sin que les expliquemos nada.

Rapidez y Fluidez: Nuestro sistema debe ser veloz y conciso. El objetivo principal de este proyecto es quitarle el estrés a la gente, por lo que el catálogo y las respuestas al interactuar con la pantalla táctil deben cargar de inmediato, evitando cualquier tiempo de espera frustrante.

Integridad de Datos (Transacciones seguras): Este es un punto crítico para evitar el caos en la bodega. Cuando el cajero marca una orden como pagada, ocurren dos cosas casi al mismo tiempo: la orden cambia de estado a "Pagada" y el inventario se descuenta. Si la red o el sistema llegaran a fallar justo después de cambiar el estado, pero *antes* de descontar el inventario, se generaría una inconsistencia grave. Para proteger la base de datos de estos errores, implementaremos estas acciones usando "transacciones". Esto asegura que, o se completan los dos pasos correctamente, o no se guarda ninguno, protegiendo así el cuadre de ventas de la tienda.

8.3 Historias de Usuario

Para el Cliente : Pensando en la persona que llega a la tienda buscando algo rápido, definimos estas historias: "Como cliente, quiero poder ver las categorías de productos en la pantalla, para encontrar lo que busco sin tener que perseguir a un vendedor". Para no frustrarlo: "Como cliente, quiero que el sistema me muestre *solo* las tallas que realmente hay en bodega". Y finalmente: "Como cliente, quiero ver el total de lo que llevo en mi carrito antes de finalizar, para asegurarme de no pasarme de mi presupuesto".

Para el Cajero: Esta persona va a estar todo el día atendiendo gente y probablemente con algo de estrés en horas pico, así que sus historias son vitales para agilizar su trabajo: " quiero ver una lista clara de los pedidos pendientes, para poder cobrar rápido y no hacer esperar a la fila". Y la más importante para su cierre de turno: " quiero que al marcar una orden como pagada el inventario se actualice solo, para no tener que estar haciendo cálculos manuales al final del día y poder irme temprano a mi casa".

Para el Administrador: Generalmente será el dueño o el gerente de la sucursal, por lo que identificamos estas necesidades prácticas: "Como administrador, quiero poder agregar productos nuevos fácilmente, para mantener el catálogo actualizado sin tener que depender de un técnico que me ayude con el sistema". Y para las ventas: " quiero poder crear promociones directamente desde el sistema, para aprovechar oportunidades de fin de semana sin complicaciones y sin gastar en imprimir carteles nuevos".

8.4 Casos de Uso

Armar Pedido (Cliente): Cuando el cliente llega a la tienda y se acerca a la pantalla interactiva, el sistema le muestra las categorías de productos. El cliente puede explorar, por ejemplo, la categoría de zapatos, y al seleccionar un modelo específico, el sistema consulta la base de datos y le muestra *solamente* las tallas y colores que realmente tienen stock disponible. Esto evita que el cliente se ilusione con algo que no puede comprar. Va agregando productos a su carrito y, cuando termina, selecciona "Finalizar Pedido". El sistema le muestra un resumen con el total, guarda la orden con el estado "Pendiente de Pago", y genera en pantalla un número de ticket (como el #105) para que pague en caja. Si el cliente intenta finalizar sin haber agregado nada, el sistema le avisa amablemente que su carrito está vacío.

Iniciar Sesión (Empleados): Tanto cajeros como administradores tienen que ingresar su usuario y contraseña en el panel web. El sistema verifica sus credenciales y, según el perfil, les muestra sus herramientas de trabajo correspondientes. Si las credenciales son incorrectas, el sistema frena el acceso y muestra un mensaje de error.

Ver Órdenes Activas (Cajero): Cuando el cajero inicia sesión, nuestro sistema le muestra una lista de todos los pedidos que los clientes han generado en la pantalla interactiva pero que todavía no han pagado. Ahí puede ver el número de orden, el monto total y la hora. Esto le permite estar preparado cuando los clientes se acercan a la caja.

Confirmar Pago (Cajero): Es el caso de uso central para las ventas. Cuando un cliente llega con su ticket, el cajero busca la orden en el sistema (por ejemplo, la #105) y verifica el detalle. Después de recibir el pago, marca la orden como pagada. En ese momento exacto, el sistema cambia el estado a "Pagada y Entregada" y automáticamente descuenta los productos vendidos del inventario. Si el número de orden no existe o ya había sido procesado, el sistema alerta al cajero.

Gestionar Productos e Inventario (Administrador): Desde su panel, el dueño o encargado puede crear productos nuevos llenando un formulario con nombre, precio, descripción y foto. También puede editar productos existentes para lanzar ofertas, o ajustar manualmente las cantidades cuando llega mercancía nueva o se detecta un error en la bodega. Lo más importante: cuando el administrador guarda cualquier cambio, nuestro sistema actualiza inmediatamente el catálogo que los clientes están viendo en la pantalla de la tienda.

Diagrama de Casos de Uso

